

Iterative:

① → ② → ③ → ④ → NULL pre = NULL
cur = head

pre
↓
NULL

curr

1 → 2 → 3 → 4 → NULL

temp = curr

temp

Diagram illustrating the linked list structure and pointer movement:

```
graph LR; pre[pre] --> node1((1)); node1 --> node2((2)); node2 --> node3((3)); node3 --> node4((4)); node4 --> null[NULL];
```

Initial state: $pre \rightarrow NULL$, $cur \rightarrow 1$, $temp \rightarrow 1$.

Iteration 1: $cur = cur \rightarrow next$ (moves to 2), $temp = temp \rightarrow next$ (moves to 2).

Iteration 2: $cur = cur \rightarrow next$ (moves to 3), $temp = temp \rightarrow next$ (moves to 3).

Iteration 3: $cur = cur \rightarrow next$ (moves to 4), $temp = temp \rightarrow next$ (moves to 4).

Iteration 4: $cur = cur \rightarrow next$ (moves to NULL), $temp = temp \rightarrow next$ (moves to NULL).

Diagram illustrating the linked list structure after the first iteration:

```
graph LR; pre((pre)) --> node1((1)); node1 --> node2((2)); node2 --> node3((3)); node3 --> node4((4)); node4 --> null((NULL)); temp((temp)) --> node1;
```

The diagram shows a linked list with nodes 1, 2, 3, and 4. The 'pre' pointer points to node 1. The 'temp' pointer also points to node 1. The 'cur' pointer points to node 2. The 'next' pointer points to node 3. The 'null' pointer points to NULL.

Diagram illustrating the linked list structure and pointer updates:

```
graph LR; NULL --> 1((1)); 1 --> 2((2)); 2 --> 3((3)); 3 --> 4((4)); 4 --> NULL; pre = temp; cur = 2
```

The diagram shows a linked list with nodes 1, 2, 3, and 4. Node 1 is the head, and the list ends at NULL. The current pointer (cur) is at node 2, and the previous pointer (pre) is at node 1. The next pointer of node 2 is node 3, and the next pointer of node 3 is node 4. The next pointer of node 4 is NULL.

```
ListNode* reverseList( head ) {
    ListNode* pre = nullptr;
    ListNode* cur = head;
    ListNode* temp;
    while ( cur ) {
        temp = cur;
        cur = cur->next;
        temp->next = pre;
        pre = temp;
    }
    return pre;
}
```

Diagram illustrating the iterative steps of reversing a linked list:

- Initial State:**
 - `pre` points to `NULL`.
 - `cur` points to node `1`.
 - `temp` points to node `2`, which points to node `3`, which points to node `4`, which points to `NULL`.
- Step 1:**
 - `cur` moves to node `2`.
 - `temp` moves to node `3`.
- Step 2:**
 - `cur` moves to node `3`.
 - `temp` moves to node `4`.
- Step 3:**
 - `cur` moves to node `4`.
 - `temp` moves to `NULL`.

A blue arrow on the left indicates the progression of the "2nd loop".

Diagram illustrating the linked list structure and pointer movement:

Initial state: $NULL \leftarrow (1) \leftarrow (2) \leftarrow (3)$ (where 3 is the *pre* pointer) and $(4) \rightarrow NULL$ (where 4 is the *curr* pointer).

After moving *pre* to *curr*: $pre = (3)$, $curr = (4)$.

After moving *curr* to *pre*: $pre = (4)$, $curr = NULL$.

Diagram illustrating the linked list structure after the recursive reversal process:

```
graph LR; NULL --> 1((1)) --> 2((2)) --> 3((3)) --> 4((4));
```

The diagram shows a linked list with nodes 1, 2, 3, and 4. The head pointer (NULL) points to node 1. The 'prev' pointer is positioned above node 4, and the 'curr' pointer is positioned above the NULL at the end of the list. A 'temp' pointer is shown below node 4, with an arrow pointing down to the text 'return prev'.